

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE1104 Fundamentos de Sistemas
Computacionales

Documentación técnica

Radar 2D

Estudiantes:

Felipe Sanabria Pacheco
Kevin Sanabria Quesada

Profesor:

Leonardo Araya Martínez

Semestre I 2026

0. Índice

Contenido

0. Índice	2
1. Introducción.....	3
1.1. Propósito del documento	3
1.2. Descripción del proyecto.....	3
1.3. Objetivos y Alcance	3
2. Descripción General del Sistema.....	4
2.1. Perspectiva del Producto	4
2.2. Funcionalidades principales	4
2.3. Restricciones del sistema, Supuestos y Dependencias	4
3. Requisitos del Sistema.....	7
3.1. Requisitos Funcionales	7
3.2. Requisitos No Funcionales	7
4. Diseño de la Solución.....	8
4.1. Visión General de la Arquitectura	8
4.2. Descripción de Componentes	8
4.4. Diagrama de Flujo y Secuencia	9
4.5. Decisiones de Diseño y Justificación.....	9
5. Implementación y Validación de Calidad	12
5.1. Estructura del repositorio	12
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución	12
5.3. Estrategias y Casos de Prueba.....	13
5.4. Resultados de pruebas y problemas encontrados	13
6.1. Herramientas de IA utilizadas	15
6.3. Declaración de Responsabilidad y Autoría.....	16
7. Conclusión.....	17
8. Bibliografía.....	18

1. Introducción

Este proyecto busca crear un que recree el funcionamiento de un radar 2D de 360° utilizando Arduino.

1.1. Propósito del documento

Este documento describe el diseño, implementación y funcionamiento del sistema Radar 2D dirigido a estudiantes y profesores.

1.2. Descripción del proyecto

El proyecto consiste en la creación de un radar 2D mediante el uso de un servomotor, un sensor ultrasónico y un Arduino

1.3 Objetivos y Alcance

Objetivo general:

Crear un radar 2D funcional utilizando un Arduino como plataforma de hardware y realizando el posterior análisis y muestra en una interfaz gráfica desarrollada en Python

Objetivos específicos:

1. Desarrollar una maqueta visualmente agradable que permita el muestreo de objetos en modo 2D utilizando un sensor de proximidad.
2. Desarrollar una interfaz gráfica donde se muestre la ubicación de los objetos identificados en tiempo real.
3. Identificar la velocidad promedio de los objetos utilizando como insumo el delta de la posición anterior versus la actual.

2. Descripción General del Sistema

El sistema consta de 2 partes, una maqueta física realizada en corte láser donde instalan el servomotor, el sensor ultrasónico, el led y la placa Arduino. Mientras que el software se encarga de toda la programación realizada para que el radar funcione y proyecte los objetos que está visualizando

2.1. Perspectiva del Producto

El sistema opera tanto en Python como en Arduino por eso depende uno de otro, estos están relacionados uno con otro ya que Python genera una interfaz gráfica la cual da señal al Arduino para que envíe la programación de como debe de funcionar el radar hacia la maqueta, además de poder registrar los datos percibidos por el sensor.

2.2. Funcionalidades principales

Enumera las funcionalidades de alto nivel del sistema (no detalle de implementación). Puede representarse como una lista o diagrama de casos de uso.

Ejemplo:

- [Registro de datos]: [El sensor ultrasónico registra los datos de distancia de distintos objetos]
- [Rotación]: [para que el radar registre los datos de todo a su alrededor este debe de girar para lograrlo]
- [Presentación de datos]: [La interfaz gráfica permite ver datos como donde esta el objeto y a que distancia se encuentra]
-

2.3. Restricciones del sistema, Supuestos y Dependencias

Restricciones

- [Aplicación usada]: El sistema solo funciona si es accionada desde la aplicación de Python conectada a la aplicación de Arduino.
- [Rango de detección]: El sistema estará limitado por el alcance efectivo del sensor ultrasónico

Supuestos:

- [Area de trabajo]: El área donde esta ubicado el radar no estará sobre poblado de objetos

Dependencias:

- [maqueta-computadora]: La parte de software del sistema debe de estar conectada a la parte de Hardware de este si no es así en sistema no funcionará
- [uso Python]: El sistema debe usara exclusivamente desde la aplicación de Python no se puede usar otra aplicación como seria Visual Studio Code

3. Requisitos del Sistema

Esta sección documenta de forma estructurada todos los requisitos que el sistema debe cumplir.

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	[Registrar datos]	Alta	[utilizar el sensor ultrasónico para registrar la distancia de un objeto]
RF-002	[Presentar datos]	alta	[Exponer los datos registrados en la interfaz gráfica]

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Disponibilidad	Debe poder funcionar de manera continua durante varias horas sin interrupciones ni bloqueos.	El hardware funciona durante un tiempo prolongado
RNF-002	Uso de hardware accesible.	El hardware, es decir las piezas utilizadas pueden ser adquiridas por cualquier persona	El hardware se puede encontrar y tiene un precio medio que hace que todos puedan acceder a él
RNF-003	Confiabilidad	Las mediciones deben manejar errores de lectura del sensor sin provocar fallos del programa.	Hay un margen de error sobre los datos presentados

4. Diseño de la Solución

4.1. Visión General de la Arquitectura

El sistema utiliza una arquitectura simple basada en Arduino UNO, con entrada (sensor), procesamiento (software) y salida (interfaz gráfica).

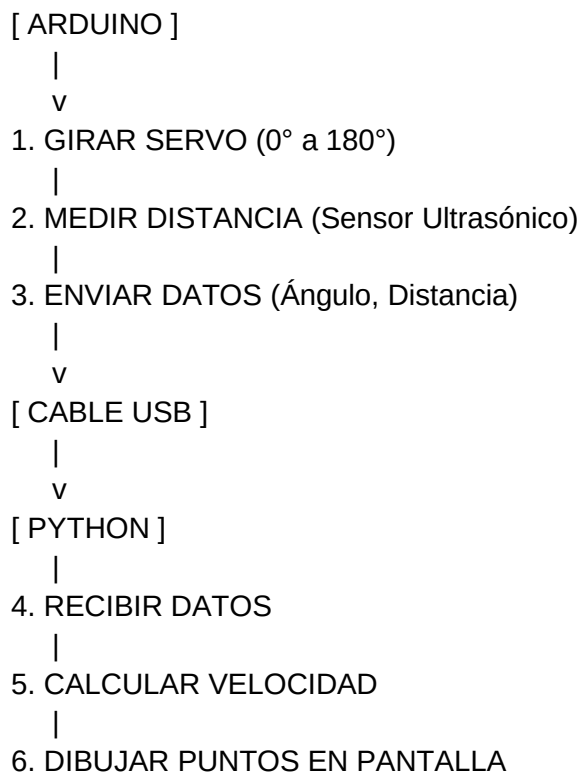
4.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
Arduino UNO	Manda y recibe las señales para encender el servomotor y el sensor ultrasónico	microcontrolador ATmega328P	Puerto USB, debe estar conectado para ser energizado
Servomotor de 360°	rotación del sensor ultrasónico	Motor DC + caja de engranajes, con circuito de control PWM (potenciometro fijo/eliminado para permitir giro continuo en vez de detenerse en un ángulo)	Señal PWM enviada por el Arduino, alimentación de 5V (o fuente externa si requiere más corriente), tierra (GND) común con el Arduino
Sensor ultrasónico	Registro de datos	Módulo emisor-receptor de ultrasonido (ej. HC-SR04), mide distancia calculando el tiempo que tarda el eco en regresar	Pines digitales Trigger y Echo conectados al Arduino, alimentación de 5V, tierra (GND) común con el Arduino
LED	Luz intermitente	Semiconductores	La intensidad de luz depende de la intensidad de corriente

4.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Lenguaje (Frontend)	Python	3.14	Interfaz grafica
Lenguaje (Backend)	Arduino	IDE 2.3.8	Programación de las funciones de Arduino(cuando encender el servomotor y el sensor ultrasónico)

4.4. Diagrama de Flujo y Secuencia



4.5. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Realización maqueta	Arduino	Arduino UNO	El proyecto solicita el uso de Arduino
Interfaz Grafica	Tkinter	Tkinter	Es la herramienta sabíamos utilizar y encontramos más eficiente para realizar esta tarea
Material	Madera acrílico	Madera	estéticamente es más agradable a la vista

5. Implementación y Validación de Calidad

5.1. Estructura del repositorio

Describe la organización de carpetas y archivos del proyecto. Explica qué contiene cada directorio principal.

Ejemplo:

```
/proyecto-raiz  /src          → Código fuente principal  /tests          → Pruebas
unitarias e integración  /docs          → Documentación del proyecto  /config          →
Archivos de configuración  /scripts        → Scripts de automatización  README.md
→ Descripción general y guía de inicio rápido
```

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Lista los prerequisites de software necesarios para ejecutar el proyecto en un entorno local de desarrollo. Además, se proporcionan instrucciones paso a paso para que cualquier persona pueda ejecutar el proyecto desde cero. Incluye comandos exactos.

Ejemplo:

- Entorno:
 - Sistema Operativo: [Windows 10]
 - Lenguaje / Runtime: [Python 3.10+]
 - Lenguaje / Runtime: [Arduino]
 -

- Guía de instalación y ejecución:

```
0 # 1. Clonar el repositorio git clone
  https://github.com/[usuario]/[repositorio].git cd [repositorio]# 2.
  Instalar dependencias[npm install / pip install -r requirements.txt]#
  3. Configurar variables de entornocp .env.example .env# Editar .env con
  los valores correspondientes# 4. Inicializar la base de datos[comando de
  migración]# 5. Ejecutar el servidor[npm start / python app.py]
```

5.3. Estrategias y Casos de Prueba

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación de funciones y métodos individuales de forma aislada en la interfaz gráfica.	Python	>= 70% del código
Pruebas Unitarias	Verificación de funciones y métodos individuales de forma aislada en la interfaz gráfica.	Arduino	Revisa que las conexiones se hayan realizado de manera correcta
Pruebas Funcionales	Validación del comportamiento del sistema desde la perspectiva del usuario final.	Python, Arduino y maqueta	Casos de uso principales

- Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	[prueba interfaz]	1. Iniciar la ejecución	Se presenta el diseño de la interfaz.	[se presentó la interfaz]	✓ Pasó
CP-002	[prueba de sensor y servo]	1. Iniciar programación en Arduino 2. Revisar que todos estén funcionando	El servomotor gira y el sensor presenta los datos	[el mismo que el esperado]	✓ Pasó
CP-003	[Interfaz y maqueta]	1. iniciar la ejecución 2. revisar que el servo gira el sensor registre datos y que la interfaz muestre los datos del sensor	[todo funcione y se presenten los datos esperados]	[el mismo que el esperado]	✓ Pasó

5.4. Resultados de pruebas y problemas encontrados

- Resultados:

Se realizaron varias pruebas hasta que cada parte del sistema funciono correctamente en conjunto
Problemas encontrados:

- Problemas Solucionados:

Difícil coordinación de la velocidad del radar virtual con la del servomotor

Momentos donde el sistema “se reinicia”; para, da una media vuelta y sigue dando vueltas (lo que nos sucedía al inicio).

Y se soluciono haciendo que el servomotor diera dos vueltas y se regrese dos vuelta s

6. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica

6.1. Herramientas de IA utilizadas

Lista todas las herramientas de IA utilizadas durante el desarrollo. Sé específico con el nombre, versión (si aplica) y propósito.

Ejemplo:

Herramienta de IA	Tipo	Período de Uso	Propósito
[ChatGPT / GPT-4]	Modelo de lenguaje (LLM)	[6/2026]	Apoyo en la creación de código, resolución de dudas técnicas.

6.2. Descripción del Uso de Inteligencia Artificial

- Generación y asistencia en código

6.3. Declaración de Responsabilidad y Autoría

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "[Radar 2D]", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusión

El proyecto cumplió con los objetivos planteados: se construyó una maqueta funcional con un sensor ultrasónico montado sobre un servomotor de giro continuo de 360°, y una interfaz gráfica en Python que muestra en tiempo real la posición y velocidad de los objetos detectados, calculada a partir del delta entre lecturas consecutivas.

Los principales retos fueron modificar el servomotor para lograr rotación continua, mantener una comunicación serial estable entre Arduino y Python, y convertir las coordenadas polares del sensor a cartesianas para graficar correctamente. Esto fortaleció conocimientos en microcontroladores, comunicación serial, interfaces gráficas y trigonometría aplicada.

En equipo, dividir el trabajo entre hardware y software, con comunicación constante y uso de GitHub, permitió integrar ambas partes sin contratiempos. Como mejora futura, se podría añadir un segundo servomotor para un radar 3D. Además, se puede mencionar posibles mejoras a futuro en proyectos similares.

8. Bibliografía